

Ejercicio Demostrativo

Nombre del Auxiliar

Tiempo estimado: 30 min

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Segundo Semestre 2026
[Laboratorio - Nombre del Curso - Sección]

1. Marco Formativo

1.1 Valores

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Responsabilidad	El estudiante aplica este valor al seguir el flujo de trabajo definido (GitFlow), asegurando que cada cambio se desarrolle en su rama correspondiente y se integre mediante pull requests, evitando mezclas directas en <code>main</code> o <code>develop</code> .

1.2 Competencias

Tipo de Competencia	Redacción
Competencia General	Aplica metodologías de control de versiones y trabajo colaborativo en la gestión de proyectos de software, asegurando integridad, trazabilidad y calidad en el ciclo de desarrollo.
Competencia Específica	El estudiante desarrolla la capacidad de implementar el flujo GitFlow en un repositorio de GitHub, creando y gestionando ramas, resolviendo integraciones mediante pull requests y etiquetando versiones estables, comprendiendo el propósito de cada paso dentro de un equipo de desarrollo.

2. Palabras Clave

GitFlow: Modelo de branching (ramificación) para Git que Define ramas principales (`main`, `develop`) y de soporte (`feature`, `release`, `hotfix`) para organizar el desarrollo.

Pull Request (PR): Mecanismo en GitHub para solicitar la integración de cambios de una rama a otra, permitiendo revisión y discusión antes de fusionar.

Rama (`branch`): Línea independiente de desarrollo que permite trabajar en funcionalidades o correcciones sin afectar el código estable.

Tag (etiqueta): Referencia fija a un commit concreto, usada comúnmente para marcar versiones publicadas o hitos del proyecto.

3. Resumen

Este ejercicio demostrativo presenta de forma guiada la implementación del flujo GitFlow sobre un repositorio en GitHub. El asistente mostrará cómo inicializar el repositorio, crear ramas `develop`, trabajar con una rama `feature`, integrar cambios mediante un pull request, simular una corrección urgente con `hotfix` y finalmente etiquetar una versión estable.

La clase debe orientarse a que el estudiante comprenda la diferenciación de propósitos entre ramas, el flujo de integración colaborativa y la importancia de los pull requests como control de calidad.

4. Contenido

Descripción del ejercicio

Se simula un pequeño proyecto: una calculadora de funciones matemáticas básicas. Partiendo de un repositorio vacío en GitHub, se implementa GitFlow para:

1. Crear la rama `develop` como base de integración.
2. Desarrollar una nueva funcionalidad (ej. suma) en rama `feature/suma`.
3. Integrar dicha funcionalidad a `develop` mediante un PR.
4. Simular una corrección urgente en la rama `main` con un `hotfix`.
5. Marcar una versión `v1.0.0` con tag y documentar el resultado final.

Objetivo de la demostración

Demostrar el procedimiento para aplicar GitFlow sobre un repositorio de GitHub, utilizando línea de comandos (Git) y la interfaz web de GitHub, con el fin de que el estudiante comprenda el ciclo de desarrollo colaborativo basado en ramas, integración mediante pull requests y versionado con tags.

Recursos necesarios

- **Software:** Git (≥ 2.30), navegador web.
- **Plataforma:** GitHub (cuenta personal o de laboratorio).
- **Herramientas técnicas:** Terminal / Git Bash, editor de texto (VS Code, Notepad++).

- **Archivos de apoyo:** Ninguno adicional (se creará un archivo `calculadora.py` durante la demo).
- **Equipo:** Computadora con conexión a internet.
- **Configuración mínima:** Git configurado con `user.name` y `user.email`. Repositorio remoto creado en GitHub (vacío, sin README por defecto).

Desarrollo paso a paso

Paso 1

Acción inicial: Crear repositorio en GitHub y clonarlo localmente.

Objetivo: Establecer el entorno base.

Qué observar: El asistente inicia sesión en GitHub, crea un nuevo repositorio (ej. `demo-gitflow`) **sin** archivos iniciales, obtiene la URL del remoto y ejecuta `git clone <url>` en la terminal. Luego muestra que solo existe la rama `main`.

Paso 2

Acción: Crear rama `develop` y subirla al remoto.

Explicación: GitFlow requiere que `develop` sea la rama de integración.

Comandos ejecutados:

```
cd demo-gitflow
```

```
git checkout -b develop
```

```
git push -u origin develop
```

Validación importante: Verificar en GitHub que aparecen ambas ramas (`main` y `develop`).

Paso 3

Ejecución principal: Crear una rama `feature/suma`, implementar funcionalidad y subirla.

Detalle: Desde `develop` se crea:

```
git checkout develop
```

```
git checkout -b feature/suma
```

Crear archivo calculadora.py con función suma

echo "def sumar(a, b): return a + b" > calculadora.py

git add calculadora.py

git commit -m "feat: agregar función suma básica"

git push -u origin feature/suma

Interpretación de resultado parcial: En GitHub aparece la nueva rama. Se debe verificar que el archivo existe y el commit tiene mensaje semántico.

Paso 4

Cierre técnico (integración y etiquetado): Crear Pull Request de **feature/suma** a **develop**, fusionar localmente y etiquetar versión.

Pasos en GitHub:

- Abrir PR desde **feature/suma** hacia **develop**.
- Revisar cambios (sin conflictos) y hacer clic en "Merge pull request".
- Eliminar rama remota **feature/suma** (opcional).

Confirmación final:

git checkout develop

git pull origin develop

git tag -a v1.0.0 -m "Versión inicial: funcionalidad suma integrada"

git push origin v1.0.0

Evidencia: En GitHub, la sección "Releases" muestra el tag **v1.0.0**.

Observaciones importantes

- Error común: Hacer commits directamente en **main** o **develop**. Siempre usar ramas de soporte (**feature**, **hotfix**).

- Buena práctica: Los nombres de rama deben ser descriptivos: **feature/nombre-funcionalidad, hotfix/descripcion.**
- Advertencia: No fusionar un PR sin verificar que los tests (si existen) pasan.
- Recomendación: Usar plantillas de mensajes de commit (Conventional Commits) para mejorar trazabilidad.

Tabla 1

Elemento	Descripción
Rama base (producción)	<code>main</code>
Rama de integración	<code>develop</code>
Rama de funcionalidad	<code>feature/suma</code>
Rama de urgencia	<code>hotfix/error-critico</code>
Comando para crear rama	<code>git checkout -b nombre-rama</code>
Mecanismo de revisión	Pull Request en GitHub

Resultado esperado

Al finalizar el ejercicio, el repositorio en GitHub debe contener:

- Ramas `main` y `develop` sincronizadas.
- Historial de commits que incluya el merge del PR desde `feature/suma` hacia `develop`.
- Un tag `v1.0.0` asociado al commit de integración en `develop`.
- El archivo `calculadora.py` con la función `sumar` disponible en `develop` (y eventualmente en `main` si se fusionó ascendente, aunque GitFlow recomienda mantener `main` solo para releases).

Aplicación práctica

Este ejercicio se relaciona directamente con el ciclo de desarrollo profesional:

- Trabajo en equipo con repositorios compartidos.
- Control de calidad mediante revisiones de código (pull requests).
- Preparación de lanzamientos usando ramas `release` (extensión opcional) y etiquetado.
- Respuesta rápida a errores en producción mediante `hotfix`.

Además, sirve como base para prácticas más avanzadas: integración continua, despliegue automático y gestión de conflictos.

5. Aconsejando al siguiente Tutor para la realización de ejemplos

Dificultades encontradas:

- Confusión inicial entre `git push`, `git pull` y la diferencia entre ramas local/remota.
- Estudiantes que olvidan cambiar de rama antes de crear una nueva (`git checkout develop` primero).
- Errores al resolver un PR con conflictos (no se aborda en este ejercicio básico, pero algunos preguntan).

Recomendaciones para futuras sesiones:

- Tener el repositorio de demostración ya preparado como respaldo en caso de fallas de internet.
- Usar proyección con terminal y navegador en vivo, destacando cada comando antes de ejecutarlo.
- Pedir a los estudiantes que repliquen en paralelo sus propios repositorios (aprendizaje activo).
- Incluir un pequeño ejercicio de conflicto en otra sesión, ya que es una necesidad real.

Qué puntos consideras fue de mayor dificultad para el estudiante:

- Comprender por qué `main` no se toca directamente: cuesta interiorizar el control de calidad.
- El concepto de *pull request* como solicitud formal, no como simple fusión.
- Recordar actualizar la rama local (`git pull`) antes de crear una nueva rama desde `develop`.

6. Referencias

1. Atlassian. (2023). *GitFlow branching model*. Recuperado de <https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow>
2. GitHub Docs. (2024). *About pull requests*. Recuperado de <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests>
3. Driessen, V. (2010). *A successful Git branching model*. nvie.com. Recuperado de <https://nvie.com/posts/a-successful-git-branching-model/>